

AI Agents Setup Guide

AI Agents Setup Guide

Everything a new team member needs to get Claude Code, Codex, and Pi working with this repo's skills, tools, and optimizations. Follow in order — each section builds on the previous.

What this repo is

A shared library of AI agent skills, tools, standards, and automation scripts. Your projects don't copy from it — they import from it via live @-references. When this repo is updated, your projects pick up changes automatically on the next session.

AI-Skills/	← this repo (shared library, clone once)
skills/	← polish, code-reviewer, security-review, ...
tools/	← ticket, handoff
standards/	← general, git
scripts/	← hook automation (handoff, polish trigger, pre-commit)
guides/	← this file and context-optimization.md
extensions/pi/	← Pi lifecycle extensions
your-project/	← your work repo
CLAUDE.md	← @-imports pointing into AI-Skills
AGENTS.md	← skill table for Codex and Pi
HANDOFF.md	← session context (auto-managed)

Step 1 — Clone AI-Skills to the standard path

The standard path matters. All @-import references and hook scripts use ~/Developer/AI-Skills. Cloning elsewhere breaks them.

```
mkdir -p ~/Developer
git clone https://github.com/sunitghub/AI-Skills.git ~/Developer/AI-Skills
```

Verify:

```
ls ~/Developer/AI-Skills/skills.sh  # should exist
```

Step 2 — Install RTK (token optimizer)

RTK filters verbose CLI output before it reaches the AI's token budget. Required for all agents.

```
brew install rtk
rtk --version  # verify
```

If rtk gain fails after install, you may have the wrong rtk package (name collision on crates.io).
Use `brew install rtk` — not `cargo install rtk`.

Steps 3–5 — Agent setup (automated)

Run the setup script — it handles Claude Code, Codex, and Pi, and is safe to run multiple times:

```
~/Developer/AI-Skills/init-agent.sh
```

It will prompt you to choose an agent (or all), back up any existing config files before modifying them (.bak extension), and report what was added vs already present.

You can also run non-interactively:

```
~/Developer/AI-Skills/init-agent.sh claude  # Claude Code only
~/Developer/AI-Skills/init-agent.sh codex  # Codex only
~/Developer/AI-Skills/init-agent.sh pi     # Pi only
~/Developer/AI-Skills/init-agent.sh all    # all three
```

What it sets up per agent:

Agent	What gets configured
Claude Code	RTK native hook (rtk hook claude) via rtk init -g --auto-patch, plus handoff + quality hooks merged into ~/.claude/settings.json
Codex	RTK instructions via rtk init -g --codex → writes ~/.codex/RTK.md + @reference in ~/.codex/AGENTS.md
Pi	Copies extensions/pi/handoff.ts to ~/.pi/agent/extensions/

Manual fallback (if you prefer to inspect before applying):

- ▶ Claude Code — manual hook setup
- ▶ Codex — manual setup
- ▶ Pi — manual setup

Step 6 — Per-project setup

Run this once per project you want to use skills in.

Register the core skills

```
cd /path/to/your-project

# Context and task management
~/Developer/AI-Skills/skills.sh add handoff
~/Developer/AI-Skills/skills.sh add ticket # if using tk for task tracking

# Coding standards (applied automatically, no invocation needed)
~/Developer/AI-Skills/skills.sh add general
~/Developer/AI-Skills/skills.sh add git

# Quality pipeline (runs after tk close, before git commit)
for s in code-simplifier code-reviewer security-review polish; do
  ~/Developer/AI-Skills/skills.sh add $s
done
```

Verify registration

```
~/Developer/AI-Skills/skills.sh status
```

You should see all registered skills listed under both `CLAUDE.md` and `AGENTS.md`.

Initialize HANDOFF.md

Tell Claude or Codex: “Initialize the handoff file” — it creates `HANDOFF.md` in the project root from the template.

Step 7 — Install tk (optional, for ticket management)

```
brew tap wedow/tools
brew install ticket
tk help    # verify
```

tk is only needed if you’re using the ticket skill and tk commands for task tracking.

Verification checklist

Run through these to confirm everything is wired up correctly.

RTK

```
rtk gain          # should show "No tracking data yet" or savings stats (not an error)
rtk git status    # should run and show compact output
```

Claude Code hooks

```
cat ~/.claude/settings.json # should contain: rtk hook claude, auto-handoff, handoff-
                             inject, auto-polish-trigger, pre-commit-check
```

Codex

```
cat ~/.codex/AGENTS.md    # should contain @RTK.md reference
```

Per-project

```
~/Developer/AI-Skills/skills.sh status    # lists registered skills
```

Staying updated

When this repo is updated with improved skills or new scripts:

```
cd ~/Developer/AI-Skills
git pull
```

That’s it for existing skills. Because your project’s `CLAUDE.md` uses live @-import references into this repo, Claude Code picks up updated skill content automatically on the next session. Hook scripts update immediately too — they’re called by path.

For newly added skills: you'll need to opt in explicitly:

```
~/Developer/AI-Skills/skills.sh add <new-skill> /path/to/your-project
```

Check what's new:

```
~/Developer/AI-Skills/skills.sh list
```

How the automation works end-to-end

See [guides/context-optimization.md](#) for a full explanation of the token optimization, session handoff, and quality pipeline — including why each piece is designed the way it is.